Project Management Plan & Agile Backlog

Project Scope Constraints:

- **Target Audience:** Internal network, <10 users (Employees of the SE group).
- **Out of Scope:** User management, encryption, external department mapping.
- **Core Priorities:** Epic 1 (Data Entry MVP) must be completed before Epic 3 (Visualization).
- **Quality Drivers:** Data Integrity, Maintainability (No overengineering).

Epic 1: MVP Data Pipeline (Core Backend)

Feature 1.1: Data Ingestion, Parsing & Export

- **Requirement #:** 1.1.1
  - **Requirement Type:** Functional / Data Ingestion
  - **Description:** The system SHALL provide a drag-and-drop or copy-paste interface to import tabular grade data (.xlsx, .ods, or clipboard).
  - **Rationale:** To serve as a lightweight collector tool that integrates data from individual exam sheets seamlessly.
  - **Fit Criterion:** A user can successfully upload an `.xlsx` or `.ods` file via drag-and-drop or paste data directly from their clipboard into the application.
- **Requirement #:** 1.1.2
  - **Requirement Type:** Functional / Data Extraction
  - **Description:** When an exam record is finalized during import, the system SHALL require and extract: First Name, Last Name, Matrikelnummer, Study Program, and Semester.
  - **Rationale:** To ensure all foundational student metadata is accurately captured for the database.

    - **Fit Criterion:** An exam record is successfully saved only if all five specified fields are present and valid; missing fields trigger a user alert.
- **Requirement #:** 1.1.3
    - **Requirement Type:** Functional / Data Cleansing
    - **Description:** The system SHALL automatically ignore predefined absence codes ("T A", "A T", and "N E").
    - **Rationale:** These codes represent uncompleted exams (did not attend / sick note) and must not be considered valid academic results.
    - **Fit Criterion:** Datasets containing these codes are imported successfully, but these specific entries are omitted from database metrics and calculations.
- **Requirement #:** 1.1.4
    - **Requirement Type:** Functional / Acronym Handling
    - **Description:** IF an unrecognized study abbreviation/acronym appears during import, the system SHALL prompt the user to accept it. Once accepted, it is applied automatically to all future imports.
    - **Rationale:** To reduce manual effort over time while maintaining data consistency.
    - **Fit Criterion:** The system prompts the user once for an unknown acronym; subsequent uploads containing the same acronym require zero user intervention.
- **Requirement #:** 1.1.5
    - **Requirement Type:** Functional / Data Export
    - **Description:** The system SHALL allow users to export the aggregated data back into an Excel sheet.
    - **Rationale:** To allow users to switch fluently between the tool and Excel for custom, ad-hoc analyses.
    - **Fit Criterion:** Clicking the export button generates a properly formatted `.xlsx` or `.csv` file containing the current dashboard view's data.
- **Requirement #:** 1.1.6
    - **Requirement Type:** Functional / Grade Formatting

    - **Description:** The system SHALL store academic grades strictly as simple floats or doubles.
    - **Rationale:** Standardizes numerical storage for accurate analytics calculation.
    - **Fit Criterion:** All valid grades (e.g., 1.3, 2.7) are cast and stored in the database as float/double data types.

Feature 1.2: Transaction & Concurrency Safety

- **Requirement #:** 1.2.1
    - **Requirement Type:** Non-Functional / Data Integrity
    - **Description:** IF a server crash occurs, the database SHALL save data either completely or not at all (Strict ACID adherence).
    - **Rationale:** To prevent partial saves, race conditions, or orphaned data when multiple teachers interact with the system.
    - **Fit Criterion:** Simulating a connection loss mid-upload results in zero new partial records being committed to the permanent database.

Epic 2: Student & Course Edge Cases

Feature 2.1: Student Record Processing

- **Requirement #:** 2.1.1
    - **Requirement Type:** Functional / Grade Retention
    - **Description:** IF a student repeats an exam for a single course, the system SHALL strictly store only the best grade achieved.
    - **Rationale:** To accurately reflect the student's highest performance for analytics without penalizing them for previous attempts.
    - **Fit Criterion:** When multiple grades for the same Matrikelnummer and Course ID are uploaded, the database retains only the lowest numerical float value.
- **Requirement #:** 2.1.2
    - **Requirement Type:** Functional / Data Quality

    - **Description:** IF a student's name changes for an existing Matrikelnummer, the system SHALL display a warning to the user before updating the record.
    - **Rationale:** To prevent accidental overwrites due to typos while accommodating legitimate name changes.
    - **Fit Criterion:** Uploading a different name for an existing Matrikelnummer pauses the import and prompts a "Confirm Name Change" modal.
- **Requirement #:** 2.1.3
    - **Requirement Type:** Functional / Scope Tracking
    - **Description:** IF a student's study program changes between exams (e.g., from "WInf" to "IMIT"), the system SHALL explicitly flag and show this in red on the UI.
    - **Rationale:** To easily identify cross-disciplinary transitions or changes in degree paths.
    - **Fit Criterion:** A user record containing multiple distinct study program acronyms triggers a red visual indicator next to the student's program tag.
- **Requirement #:** 2.1.4
    - **Requirement Type:** Constraint / Exclusion
    - **Description:** Master's and Bachelor's theses SHALL NOT be recorded or aggregated in the current system.
    - **Rationale:** Explicitly removed from MVP scope by stakeholder directive to simplify the data model.
    - **Fit Criterion:** Uploading a row identified as a thesis does not store data in the standard exam tables.

Feature 2.2: Course Catalog & Archiving

- **Requirement #:** 2.2.1
    - **Requirement Type:** Functional / Course Management
    - **Description:** The system SHALL treat courses with identical names but different credits (e.g., 8 credits vs. 6 credits) as entirely separate courses.

    - **Rationale:** Credit differences signify distinct module requirements, even if the title is the same.
    - **Fit Criterion:** The database generates unique primary keys for "Course A (6 ECTS)" and "Course A (8 ECTS)".
- **Requirement #:** 2.2.2
    - **Requirement Type:** Functional / Course UI
    - **Description:** The system SHALL display the full, unabbreviated name of a course when a user hovers over an abbreviated course name.
    - **Rationale:** Course names are often heavily abbreviated in Excel; this ensures readability without cluttering the screen.
    - **Fit Criterion:** Hovering over a course acronym triggers a tooltip containing the full string name.
- **Requirement #:** 2.2.3
    - **Requirement Type:** Functional / Manual Entry
    - **Description:** The system SHALL allow users to manually add new courses to the catalog.
    - **Rationale:** Avoids reliance on hardcoded module handbooks and allows flexibility for the SE group.
    - **Fit Criterion:** A specific UI button allows a user to input a course name, abbreviation, and ECTS value to create a new valid course entity.
- **Requirement #:** 2.2.4
    - **Requirement Type:** Functional / Archiving
    - **Description:** IF a student has registered no exam activity for 4 or more years, the system SHALL automatically hide them from the overview dashboards.
    - **Rationale:** To prevent data cluttering from inactive or exmatriculated students.
    - **Fit Criterion:** A query filtering for active students automatically excludes any Matrikelnummer with a latest exam date older than 48 months.
- **Requirement #:** 2.2.5
    - **Requirement Type:** Constraint / Department Scope

    - **Description:** The system SHALL only process and map courses offered by the designated department.
    - **Rationale:** The tool cannot map a student's full academic path 1:1, so it restricts its domain to local department exams only.
    - **Fit Criterion:** Only department-specific modules are accepted and visualized.

Epic 3: Visualization & Dashboard (Core Frontend)

Feature 3.1: Dashboard UI & Matrix

- **Requirement #:** 3.1.1
    - **Requirement Type:** Non-Functional / Maintainability
    - **Description:** The frontend SHALL utilize a clean, white UI theme and rely on standard, highly maintainable frameworks, explicitly dropping visually complex components.
    - **Rationale:** Maintainability is the primary quality driver. Complex visual components degrade long-term code health if developers leave.
    - **Fit Criterion:** UI framework passes SonarQube maintainability checks; no custom, highly-nested CSS/JS visual graphs are used if a standard library alternative exists.
- **Requirement #:** 3.1.2
    - **Requirement Type:** Functional / View Architecture
    - **Description:** The core analysis views SHALL be aggregated per semester, rather than natively per course.
    - **Rationale:** To align with the stakeholder's goal of tracking semester-over-semester trends.
    - **Fit Criterion:** The default dashboard view displays aggregate metrics grouped by WiSe/SoSe chronologically.

Feature 3.2: Aggregated Analytics & Plots

- **Requirement #:** 3.2.1
    - **Requirement Type:** Functional / Filtering

    - **Description:** The system SHALL allow users to filter dashboard views by Semester and by Course, showing the best grade achieved.
    - **Rationale:** Allows exploratory data analysis for specific exam sessions or specific modules.
    - **Fit Criterion:** Selecting a specific course from a dropdown updates the dashboard to show only records containing that Course ID.
- **Requirement #:** 3.2.2
    - **Requirement Type:** Functional / Trend Tracking
    - **Description:** The system SHALL track and visualize whether specific courses are getting easier or harder over time.
    - **Rationale:** To identify bottlenecks or shifts in grading strictness.
    - **Fit Criterion:** The UI plots a course's average grade progression chronologically across multiple semesters.
- **Requirement #:** 3.2.3
    - **Requirement Type:** Functional / Failure Metrics
    - **Description:** The system SHALL calculate and display the percentage of 5.0 (Fail) grades compared strictly to the number of students who passed.
    - **Rationale:** Provides a direct ratio of success vs. failure for difficulty tracking.
    - **Fit Criterion:** The displayed percentage accurately reflects the mathematical ratio: (Count of 5.0s) / (Count of grades < 5.0).
- **Requirement #:** 3.2.4
    - **Requirement Type:** Functional / Success Metrics
    - **Description:** The system SHALL calculate and display the average grade exclusively among the students who passed.
    - **Rationale:** Including 5.0 grades artificially inflates the average, misrepresenting the performance of successful students.
    - **Fit Criterion:** The calculation function only queries database rows where the float value is between 1.0 and 4.0.

Feature 3.3: Scope Specifics

- **Requirement #:** 3.3.1
  - **Requirement Type:** Functional / Target Audience Analytics
  - **Description:** The system SHALL tailor detailed, in-depth evaluation dashboards specifically for Software Engineering (SE) Master's students, while still keeping basic tracks of all department students.
  - **Rationale:** The primary beneficiaries of this analytics tool are the handlers of the SE Master's program.
  - **Fit Criterion:** A dedicated "SE View" exists that isolates data strictly for students holding the SE Master's study program acronym.